

Protección de servicios con un WAF para The Store

Autores: Mauro Vella · Enrique Castillo · Federico Inti García Lauberer

Parte 1 · El problema: qué tenía The Store y por qué era grave

1.1 Contexto

The Store es una aplicación de e-commerce construida con **cinco microservicios** (Java/Spring, Go/Gin, Node.js/NestJS) que se despliega sobre un cluster local de Kubernetes (Kind) de un único nodo. Toda la entrada externa pasa por un único punto: `ingress-nginx` escuchando en el puerto 80/443 del host. Los backends (`catalog`, `carts`, `orders`, `checkout`) son `ClusterIP` y no deberían ser accesibles desde afuera; sólo la `ui` está publicada.

El problema de fondo: **la app no tenía ninguna defensa en la capa 7**. No había inspección de requests, ni rate limiting, ni filtrado de patrones de ataque, ni headers de seguridad. Cualquiera con `curl` podía atacar la superficie HTTP sin fricción. Y como es una arquitectura **multi-lenguaje**, corregir cada agujero en Java, Go y Node por separado significa cambios distribuidos en tres stacks distintos — lento y propenso a inconsistencias.

1.2 Hallazgos de la auditoría y su gravedad

Auditamos la app pre-WAF contra `http://localhost` y confirmamos **10 vectores explotables sin autenticación**, agrupados en **6 clases de hallazgo**:

ID	Clase de hallazgo	Severidad	OWASP 2025	Por qué es grave
H1	SSRF + path traversal vía <code>/proxy/**</code>	Crítica	A03 / A10	El endpoint <code>/proxy/<svc>/...</code> reenvía requests a los backends internos sin validar el path. Permite alcanzar rutas administrativas (<code>/actuator</code> , <code>/debug</code>) y, con <code>..</code> , salir del scope del servicio (traversal) — pivote hacia la red interna del cluster.
H2	Inyecciones en checkout (XSS / SQLi / CRLF)	Alta	A03	El formulario de checkout acepta payloads sin sanitización suficiente. XSS reflejado y SQLi llegan crudos al backend; clásico vector de robo de sesión y manipulación de datos.
H3	Endpoint de chat sin validación / prompt injection	Alta	A04	<code>POST /chat/submit</code> (feature opcional) no valida la entrada — superficie para prompt injection si el chat está activo.
H4	Spring Actuator expuesto	Alta	A05	<code>/actuator/prometheus</code> , <code>/info</code> , <code>/metrics</code> , <code>/env</code> están publicados al exterior. <code>prometheus</code> solo filtra ~19 KB de métricas operativas (memoria, threads, endpoints, versiones) — un mapa completo para el atacante.
H5	Confianza incondicional en <code>X-Forwarded-*</code>	Media	A07	nginx confía en los headers <code>X-Forwarded-For/Proto/Host</code> entrantes, permitiendo spoofear IP de origen, esquema y host.
H6	Sin rate limit / scanner detection / headers	Alta	A04 / A05	Sin límite por IP (enumeración, scraping, fuerza bruta sin freno), sin detección de scanners (<code>sqlmap</code> , <code>nikto</code> operan a cara descubierta) y 0/6 headers de seguridad (HSTS, X-Frame-Options, CSP, etc.).

El más representativo — y el que usamos como caso estrella — es **H4**: `GET /actuator/prometheus` devolvía `200 OK` con ~19 KB de telemetría interna a cualquier cliente anónimo. Es information disclosure de manual (CWE-200) y la base de reconocimiento para todo lo demás.

Parte 2 · El enfoque propuesto

2.1 La idea: un WAF en el punto de entrada

El enfoque propuesto fue desplegar **ModSecurity v3** (motor de inspección HTTP que registra o bloquea requests según reglas) junto con el **OWASP Core Rule Set (CRS)** (~reglas mantenidas por OWASP que cubren los ataques web comunes) **dentro del `ingress-nginx-controller` ya presente en el cluster**.

Cómo funciona en la práctica: cada request entrante pasa primero por ModSecurity. El motor parsea URL, headers y body, acumula un puntaje de sospecha según cuántas reglas matchean y, si supera el umbral, devuelve **403 Forbidden** en lugar de reenviar al backend. **Los cinco microservicios no se enteran del cambio**: la política vive entera en configuración del Ingress, no en el código de negocio. Un solo lugar para auditar y ajustar.

2.2 Etapas de despliegue

1. **DetectionOnly** (solo registrar): el WAF anota en el log qué *habría* bloqueado, sin bloquear. Sirve para descubrir y corregir falsos positivos sobre tráfico legítimo antes de activar el bloqueo real.

2. **On** (bloquear): una vez tuneado, el WAF pasa a responder 403 sobre las requests maliciosas.

2.3 Alcance comprometido

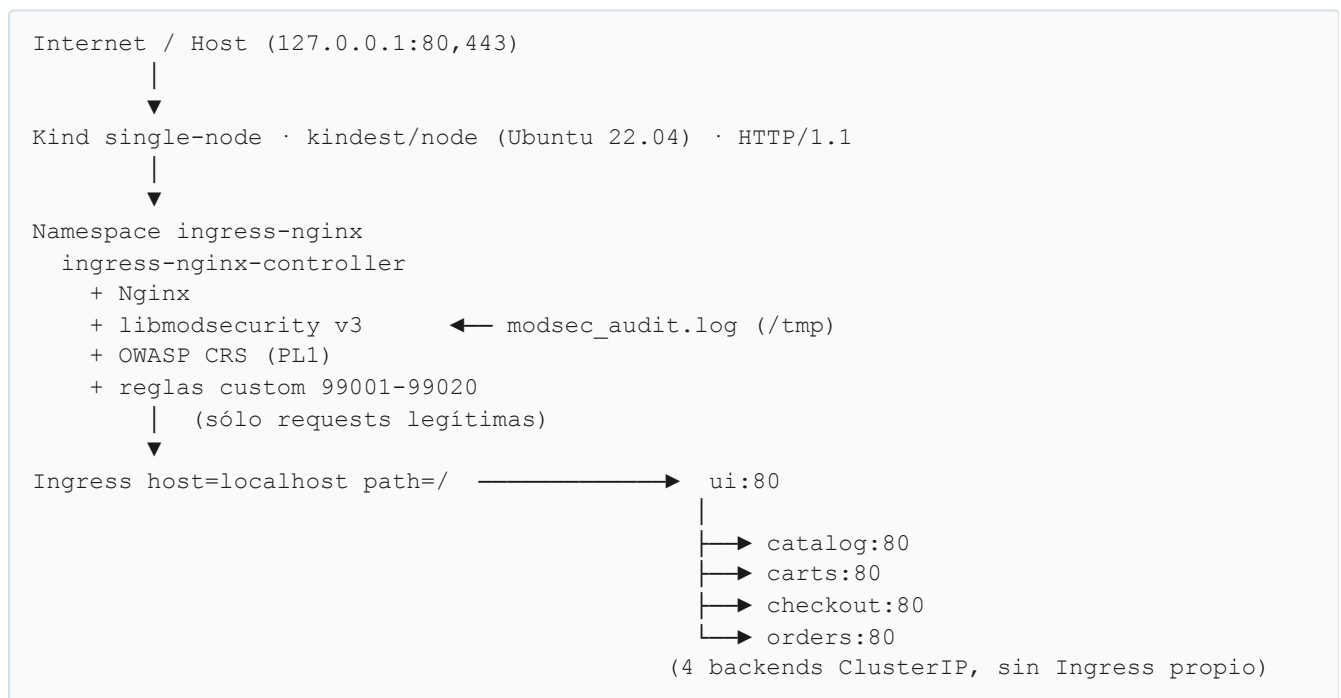
El alcance que el diseño se propuso cubrir:

- **Bloquear los endpoints de diagnóstico sensibles:** `/actuator/info`, `/actuator/metrics`, `/actuator/prometheus` pasan de 200 OK a 403 Forbidden.
- **Cerrar el agujero del proxy:** denegar cualquier request a `/proxy/*` que contenga `..` en el path o que no traiga un header de sesión válido (path traversal / SSRF, cubierto por CRS 930xxx + regla custom).
- **Detectar y bloquear User-Agents** de herramientas automáticas (`sqlmap`, `nikto`, `nuclei`) mediante CRS 913xxx + regla custom.
- **Límite de velocidad por IP** aplicado desde el Ingress sobre el tráfico externo.
- **Seis headers de seguridad HTTP** en las respuestas servidas por el Ingress. La Content-Security-Policy se entregaría en modo Report-Only / permisiva mientras se valida compatibilidad con la UI.
- **Reglas custom encima del CRS base** para la lógica específica de The Store (Actuator, proxy, scanners) — lo que el CRS genérico no puede conocer.

El alcance del cambio quedaba acotado a archivos de configuración (ConfigMap + anotaciones del Ingress). **Cero cambios en el código Java, Go o Node.**

Parte 3 · El diseño de la solución: cómo funciona y por qué cumple

3.1 Arquitectura de red



Bloque	CIDR	Rol
Host loopback	127.0.0.1/32	El usuario accede a The Store por 80/443
Docker / nodo Kind	172.18.0.0/16	Red del contenedor del nodo
Pods	10.244.0.0/16	Comunicación pod-a-pod
Services (ClusterIP)	10.96.0.0/12	IPs internas de los Services
CoreDNS	10.96.0.10	DNS interno del cluster

El WAF se inserta en el límite **norte-sur** (Internet → cluster). El tráfico **este-oeste** (entre pods, p. ej. `ui` → `catalog`) NO pasa por el WAF: es una limitación consciente del diseño (ver §3.7).

3.2 Componentes de la solución

Se desplegó **ModSecurity v3 + OWASP CRS** dentro del `ingress-nginx-controller`, sin modificar ni una línea de los microservicios. La imagen oficial `registry.k8s.io/ingress-nginx/controller:v1.13.x` ya trae `libmodsecurity` y el CRS pre-compilados; se activan vía ConfigMap. La política se compone de cuatro piezas:

- **ConfigMap del controller** — activa `enable-modsecurity` y `enable-owasp-modsecurity-crs`, define el motor (`SecRuleEngine On`), el audit log y el `Include` de las reglas custom.
- **OWASP CRS (Paranoia Level 1)** — cubre traversal, scanners e inyecciones genéricas de capa 7 (XSS, SQLi) sin tocar el código.
- **Reglas custom 99001-99020** — para la lógica específica de The Store: Actuador, traversal vía `/proxy/**`, rutas admin de backends y scanners.
- **Anotaciones del Ingress** — `limit-rps/limit-connections` (rate limiting), `more_set_headers` (6 headers de seguridad) y CSP en modo Report-Only.

3.2.1 Manifiestos

Todo el WAF vive en `deploy/waf/` y es **declarativo, idempotente y reversible**.

Archivo	Qué hace
01-controller-configmap.yaml	Reemplaza la ConfigMap del controller: activa ModSec + CRS, fuerza 429 en rate limit, <code>server-tokens: false</code> , <code>use-forwarded-headers: false</code> (cierra H5), motor <code>SecRuleEngine On</code> , audit log en <code>/tmp/modsec_audit.log</code> (<code>SecAuditLogParts ABCFHZ</code>) e <code>Include</code> de las reglas custom.
02-the-store-ingress.yaml	Reemplaza el Ingress de <code>ui</code> : rate limit por IP (<code>limit-rps: 10</code> , <code>limit-connections: 20</code> , <code>limit-burst-multiplier: 3</code>), <code>proxy-body-size: 1m</code> , los 6 headers de seguridad vía <code>more_set_headers</code> , CSP Report-Only y refuerzo de inspección de body por-Ingress.
03-controller-deployment-patch.yaml	Strategic-merge patch que monta la ConfigMap <code>modsec-custom-rules</code> como volumen read-only en <code>/etc/nginx/modsecurity-custom/</code> .
rules/the-store.conf	Las 7 reglas custom (99001-99020).
install.sh / uninstall.sh	Instalador idempotente (con smoke-test) y revert al estado pre-WAF.

3.2.2 Reglas custom (rango 99000-99999, reservado por CRS para el usuario)

ID	Hallazgo	Acción	Detalle
99001	H4 · Actuator	deny 403	Bloquea <code>/actuator/{info,metrics,prometheus,env,heapdump,...}</code> directo y vía proxy, más el índice <code>/actuator</code> (que lista en HAL todos los endpoints disponibles). Excluye <code>health</code> (lo usan los probes de K8s).
99002	H1 · traversal	deny 403	<code>/proxy/*</code> con <code>..</code> , <code>%2e%2e</code> o <code>%252e%252e</code> (doble URL-encode).
99003	H1 · admin vía proxy	deny 403	<code>/proxy/<svc>/</code> (<code>actuator debug management admin</code>).
99004	H1 · proxy sin sesión	deny 403	<code>/proxy/*</code> con cookie <code>SESSIONID</code> presente pero sin formato UUID válido (chain).
99005	H1 · proxy sin sesión	deny 403	<code>/proxy/*</code> sin ningún header <code>Cookie</code> (chain con <code>&...@eq 0</code>). Cubre el acceso directo por curl/scanner, que 99004 no ve porque libmodsecurity v3 no itera sobre una colección ausente.
99010	H6 · scanners	deny 403	User-Agents de <code>sqlmap</code> , <code>nikto</code> , <code>nuclei</code> , <code>nmap</code> , <code>wpscan</code> , ... (refuerza CRS 913xxx).
99020	tuning	pass	Whitelist quirúrgica: desactiva CRS 920350 (Host numérico) sólo para <code>Host: localhost/127.0.0.1</code> .

Las inyecciones de H2 (XSS/SQLi) **no necesitaron regla custom**: las cubre el CRS base (familias 941xxx, 942xxx, scoring 949110). Esto valida la decisión de diseño: **CRS para lo genérico + reglas custom para lo específico de la app**.

3.2.3 Integración en `local.sh`

El WAF se integró en el orquestador raíz siguiendo el estilo existente:

- `./local.sh install-waf` y `./local.sh uninstall-waf` — wrappers que validan cluster y namespace antes de delegar en `deploy/waf/install.sh` (o `uninstall.sh`).
- `./local.sh create-cluster --with-waf` — levanta el cluster, despliega la app y deja el WAF instalado en un solo comando. El WAF se instala **después** de los e2e tests a propósito: el rate limit (10 rps/IP) podría throttlear el tráfico legítimo de la suite.

3.3 Cómo el diseño soluciona cada hallazgo

Hallazgo	Cómo lo soluciona el WAF	Resultado medido
H1 · SSRF / traversal <code>/proxy/**</code>	Reglas 99002 (<code>.. /encoding</code>) + 99003 (rutas admin) + 99004/99005 (exige cookie de sesión válida). El front nunca llama <code>/proxy/*</code> , así que sólo se corta el acceso directo de un atacante.	200 → 403
H2 · XSS / SQLi en checkout	CRS base: 941xxx (XSS, libinjection), 942xxx (SQLi), scoring 949110. Sin regla custom.	200 → 403
H4 · Actuator expuesto	Regla 99001: bloquea endpoints sensibles y el índice <code>/actuator</code> , preservando <code>/actuator/health</code> para los probes de K8s.	200 (~19 KB) → 403
H5 · X-Forwarded-* spoofeable	<code>use-forwarded-headers: false</code> en el ConfigMap: nginx ignora los XFF entrantes.	vector cerrado
H6 · scanners	Regla 99010 + CRS 913xxx: bloquea User-Agents de herramientas conocidas.	200 → 403
H6 · sin rate limit	Anotaciones <code>limit-rps: 10</code> + <code>limit-burst-multiplier: 3</code> (nginx <code>limit_req</code> , leaky bucket), respuesta 429.	100× 200 → mayoría 429
H6 · sin headers	<code>more_set_headers</code> aplica los 6 headers en todas las respuestas (incluidas 4xx/5xx).	0/6 → 6/6

3.4 Verificación: por qué cumple con lo propuesto

3.4.1 Antes / después de los casos comprometidos

Corrida end-to-end real sobre Kind. Baseline pre-WAF y verificación post-WAF capturados contra `http://localhost`. Evidencia versionada en `pre-analysis/evidencias/post-waf/` (incluye el audit log de ModSecurity).

#	Caso	Pre-WAF	Post-WAF	Qué lo bloqueó
1	GET /actuator/prometheus	200 OK · ~19 KB filtrados	403	regla custom 99001
2	GET /actuator (índice HAL)	200 OK	403	regla custom 99001
3	Path traversal /proxy/catalog/../../../../actuator/info	200 OK	403	regla custom 99002
4	GET /proxy/carts/test sin cookie de sesión	200 OK (fuga datos)	403	reglas custom 99004 / 99005
5	User-Agent: Nikto / sqlmap / nuclei	200 OK	403	regla custom 99010
6	Burst de 100 requests concurrentes a /	100/100 → 200	mayoría → 429	nginx limit_req (10 rps + burst×3)
7	Headers de seguridad en la home	0/6 presentes	6/6 presentes	annotation more_set_headers

Todos los casos pasan de vulnerable a mitigado, y el tráfico legítimo se preserva: GET / y GET /actuator/health siguen respondiendo 200.

3.4.2 Suite automatizada

pre-analysis/tests/02-post-waf-attacks.sh (espejo del test pre-WAF con las assertions invertidas) verifica bloqueos y que la app legítima siga viva:

```
RESUMEN · TESTS POST-WAF
Pasaron: 27 / 27
Fallaron: 0 / 27
```

3.4.3 Defensa en profundidad confirmada en el audit log

Origen	Regla(s)	Qué cazó
Custom	99001	/actuator/{info,metrics,prometheus,env} + índice
Custom	99002	traversal .. / %2e%2e sobre /proxy/*
Custom	99003	/proxy/<svc>/actuator (ruta admin vía proxy)
Custom	99004/99005	/proxy/* sin cookie SESSIONID válida (acceso directo curl/scanner)
Custom	99010	User-Agents de sqlmap / nikto / nuclei
OWASP CRS	941100/110/160/390	XSS (libinjection + filtros) en checkout
OWASP CRS	942100/350	SQL injection (libinjection) en checkout
OWASP CRS	949110	Anomaly score excedido (suma de scores parciales)

3.4.4 Trazabilidad: propuesta → implementación

Cada objetivo de la propuesta tiene su correlato en la implementación y un test que lo valida:

Objetivo propuesto	Implementado en	Test que lo valida
Bloquear <code>/actuador/{info,metrics,prometheus}</code>	regla 99001	<code>02-post-waf-attacks.sh</code> H4.a/c/d + H4.a2 (índice)
<code>/proxy/*</code> con <code>..</code> → bloqueado	regla 99002	H1.a/c/d
<code>/proxy/*</code> sin sesión válida → bloqueado	reglas 99004/99005	H1.e/f/g
Rutas admin de backend vía proxy	regla 99003	H1.b
User-Agents de scanners	regla 99010 + CRS 913xxx	H6.a/b/c
Inyecciones XSS / SQLi	CRS 941xxx / 942xxx	H2.a/b
Rate limit por IP	anotaciones <code>limit-*</code>	test de burst (76/100 → 429)
6 headers de seguridad (CSP Report-Only)	<code>more_set_headers</code>	test de headers (6/6)
Sin tocar código de los microservicios	sólo <code>deploy/waf/</code> (config)	—

3.5 Problemas encontrados durante el tuning

Lo que costó hacer funcionar — y cómo se resolvió. El aprendizaje real del trabajo.

1. **Comillas simples rompían el parseo de nginx.** `ingress-nginx` envuelve el `modsecurity-snippet` en `modsecurity_rules '...'` (comillas simples). Cualquier `'` dentro de una regla (p. ej. `msg: '...'`) cerraba el string y rompía nginx. **Solución:** mover las reglas custom a un archivo aparte (`rules/the-store.conf`) montado como volumen y cargado con `Include`.
2. **libmodsecurity v3 es quisquilloso con las continuaciones de línea.** Las reglas multi-línea con `\` a veces cortaban la lista de acciones a la mitad. **Solución:** una `SecRule` por línea física (largas pero a prueba de balas).
3. **Falso positivo del CRS sobre Host numérico.** En el POC local el `Host` es `127.0.0.1/localhost`, y la regla CRS 920350 disparaba sobre tráfico legítimo. **Solución:** regla custom 99020 que hace `ctl:ruleRemoveById=920350` sólo cuando el `Host` es `localhost` — una whitelist quirúrgica, no un apagado global.
4. **El rate limit "no funcionaba"... hasta mandar requests concurrentes.** Un loop secuencial de curls queda por debajo de 10 rps y nunca dispara el 429. **Solución:** lanzar el burst con concurrencia real (`xargs -P 50`).
5. **El traversal daba 404 en vez de 403.** `curl` normaliza el `../` del lado cliente antes de enviarlo. **Solución:** usar `curl --path-as-is` para que el `..` viaje crudo y dispare la regla 99002.
6. **El header Cookie ausente no disparaba la regla de sesión.** `libmodsecurity v3` no itera sobre una variable de colección ausente, así que un `!@rx` sobre `REQUEST_HEADERS:Cookie` no se ejecuta cuando no hay ningún Cookie — justo el caso del atacante con `curl` directo. **Solución:** separar en dos reglas — 99004 (cookie presente pero inválida, con `!@rx`) y 99005 (header ausente, con `&REQUEST_HEADERS:Cookie @eq 0`).

7. **No romper la UI con la CSP.** Una `Content-Security-Policy` estricta rompía estilos/scripts inline de la UI. **Solución:** entregarla en modo **Report-Only**; el clickjacking igual queda cubierto por `X-Frame-Options`.
8. **No tumbar el pod del UI.** Un bloqueo ciego de `/actuator/*` también bloqueaba `/actuator/health`, que K8s usa para los probes — bloquearlo reiniciaría el pod en loop. **Solución:** la regla `99001` excluye `health`.

3.6 Demostración en vivo

El repo incluye una **demo web** (`deploy/waf/demo-web/`) que levanta una página local desde la que se pueden lanzar los ataques contra el cluster, ver el código de cada uno, su output real (status + respuesta), y togglear el WAF para repetir la suite con el firewall levantado. Es la versión interactiva de los scripts `pre-analysis/tests/01-pre-waf-attacks.sh` y `02-post-waf-attacks.sh`. Ver `deploy/waf/demo-web/README.md` para levantarla.

3.7 Limitaciones conocidas y trabajo futuro

- **Un WAF no reemplaza código seguro.** Es defensa en profundidad. Las vulnerabilidades de fondo (el `ProxyController` sin sanitizar, la validación parcial del checkout) siguen en el código; el WAF las contiene en el perímetro.
- **Sólo cubre el tráfico norte-sur.** El tráfico este-oeste entre pods no pasa por el Ingress. Mitigarlo requeriría un service mesh (mTLS + sidecars) o NetworkPolicies.
- **La sesión del proxy no es criptográfica.** Las reglas 99004/99005 exigen una cookie `SESSIONID` con formato UUID; un atacante puede fabricar una. Validar la sesión de verdad es responsabilidad de la app — el WAF eleva la barrera y cubre el caso por defecto (curl/scanner sin cookie).
- **Bypass por encoding.** Doble/triple URL-encode, Unicode raro o JSON con keys inesperadas son un campo activo de evasión de WAFs basados en firmas.
- **Falsos positivos a Paranoia Levels altos.** Quedamos en PL1 para la demo; PL2+ requeriría una ronda de tuning más larga sobre los endpoints legítimos.
- **CSP en Report-Only** observa pero no bloquea; pasar a enforcement requiere validar antes que la UI no rompa.
- **Ataques de lógica de negocio** (abusar de un cupón N veces) un WAF de reglas no los detecta; requieren reglas de negocio o bot management.

Trabajo futuro: subir a PL2 con tuning, evaluar **Coraza** (sucesor de ModSec en Go, compatible con CRS), agregar NetworkPolicies para el este-oeste y enforcement real de la CSP.

Conclusión

El proyecto cumple el criterio de éxito propuesto: **los vectores hoy explotables en el punto de entrada HTTP quedaron detectados o bloqueados por el WAF, sin romper el acceso normal** a la home, el catálogo, el carrito ni el checkout (`27/27` tests post-WAF en verde, `/` y `/actuator/health` siguen en `200`).

La decisión arquitectural clave — **ModSecurity + CRS en el Ingress, con reglas custom sólo para la lógica propia de The Store** — se validó en la práctica: el CRS resolvió las inyecciones genéricas (XSS/SQLi) sin una sola línea custom, mientras que las 7 reglas propias cubrieron lo que el CRS no

conoce (Actuator, proxy traversal, proxy sin sesión, scanners). Todo quedó **declarativo, idempotente y reversible**, integrado en `local.sh` y reproducible desde cero siguiendo el [HOWTO](#).

El mayor aprendizaje no fue escribir reglas, sino **operar el WAF**: el tuning (comillas, falsos positivos, no tumbar los probes, no romper la UI) es donde se juega que un WAF sea útil en vez de un estorbo.